1(a)-(b)(ii). Data, imputation and feature statistics

DSCI 552 Homework 6: code, answers and actual full-run results for Q1-6. Common functions are defined once; seed 552.

```
from pathlib import Path
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn import metrics as M
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier
SEED = 552
RUN_TRAINING = False  # Layout only; full notebook execution performs every fit.

def read_aps(split):
    path = Path('Homework 6 Data') / f'aps_failure_{split}_set.csv'
    with path.open() as stream:
        h = next(i for i, s in enumerate(stream) if s.startswith('class,'))
    frame = pd.read_csv(path, skiprows=h, na_values=['na'])
    assert frame['class'].isin(['pos', 'neg']).all()
    return (frame.drop(columns='class').astype(float),
            frame['class'].map({'neg': 0, 'pos': 1}).to_numpy())
X, y = read_aps('training')
T, yt = read_aps('test')
assert X.shape == (60000, 170) and T.shape == (16000, 170)
assert X.columns.equals(T.columns)
imp = SimpleImputer(strategy='median', keep_empty_features=True)
A = imp.fit_transform(X)
B = SimpleImputer(strategy='median', keep_empty_features=True).fit_transform(T)
Bt = imp.transform(T)  # Sensitivity only: train-fitted imputation.
D = pd.DataFrame(A, columns=X.columns)
cv = D.std(ddof=1) / D.mean().replace(0, np.nan)
top = cv.nlargest(13).index.tolist()
```

Official split	Shape	neg / pos	Missing cells
Training	60000 x 170	59000 / 1000	850015
Test	16000 x 170	15625 / 375	228680

Both headers are on line 21 after 20 preamble lines; na denotes missingness. The 170 predictors are numeric float64, identically ordered, and exclude class. Median imputation retains rows and resists extreme counters; methods and the footnote convention are explained on page 5.

All 170 CVs use median-imputed training rows and sample SD (ddof=1). Zero means give undefined CVs and are excluded from ranking; all-missing columns are retained with zero fallback and flagged. Neither occurs here. Main A/B imputation is separate; Bt is train-only sensitivity. CV fits preprocessing only on its training fold.

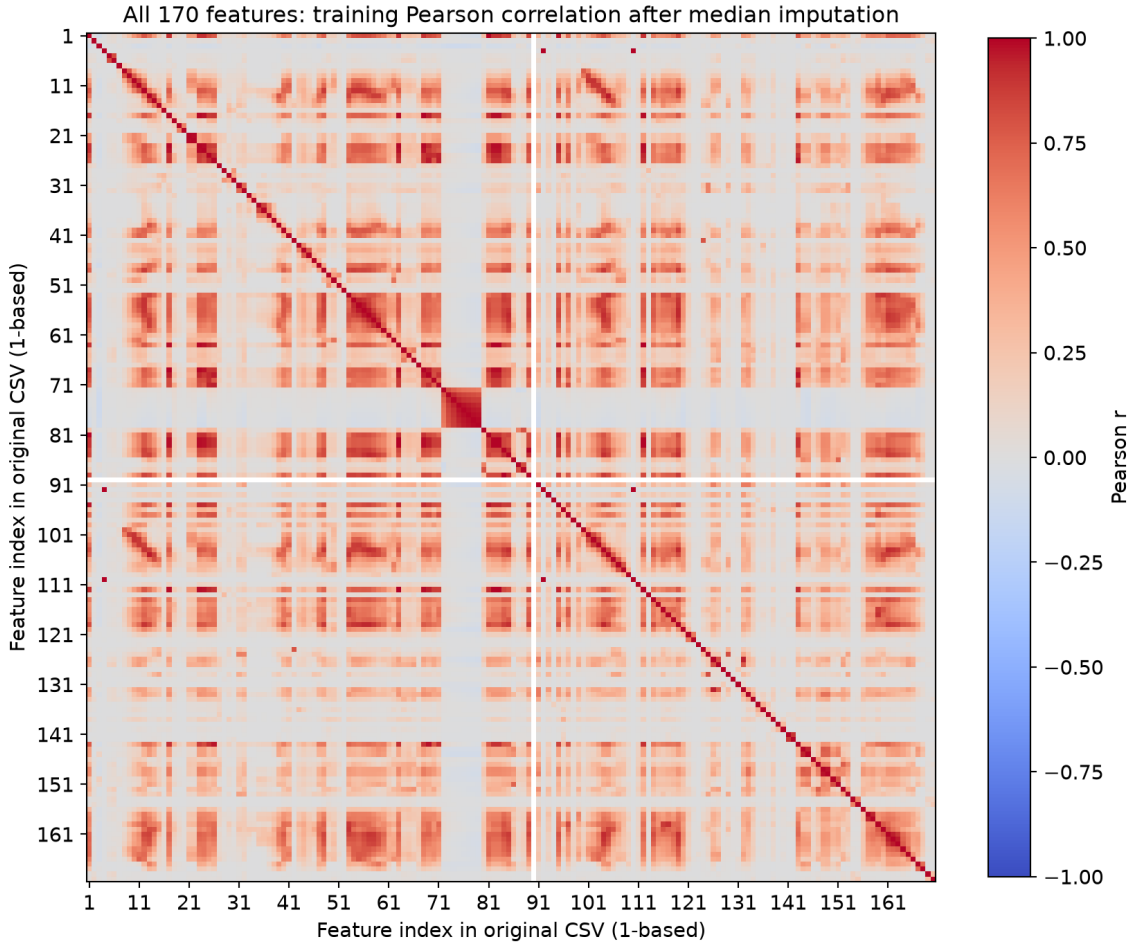
```
Top 13 CV features (visualization only):
cf_000: 244.8875 | co_000: 244.5074 | ad_000: 244.3228 | cs_009: 237.9306
dh_000: 123.2161 | dj_000: 117.4942 | ag_000: 92.9178 | as_000: 87.3325
ay_009: 84.7337 | ak_000: 80.4250 | az_009: 77.8385 | ch_000: 77.4539
au_000: 68.8828
```

CV = sample SD / mean. All 170 predictors enter modeling; the seeded sample below is for page 3 only.

```
rng = np.random.default_rng(SEED)
idx = np.r_[rng.choice(np.flatnonzero(y == 0), 2500, replace=False),
            np.flatnonzero(y == 1)]
Z = np.log1p(D.iloc[idx][top])
ys = y[idx]
colors = np.where(ys == 1, '#d34c49', '#3378a6')
```

1(b)(iii). Correlation and shared scatter-matrix code

```
corr = D.corr()
fig, ax = plt.subplots(figsize=(9, 8))
im = ax.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)
ax.set(xticks=np.arange(0, 170, 10), yticks=np.arange(0, 170, 10))
fig.colorbar(im, ax=ax, label='Pearson correlation')
```

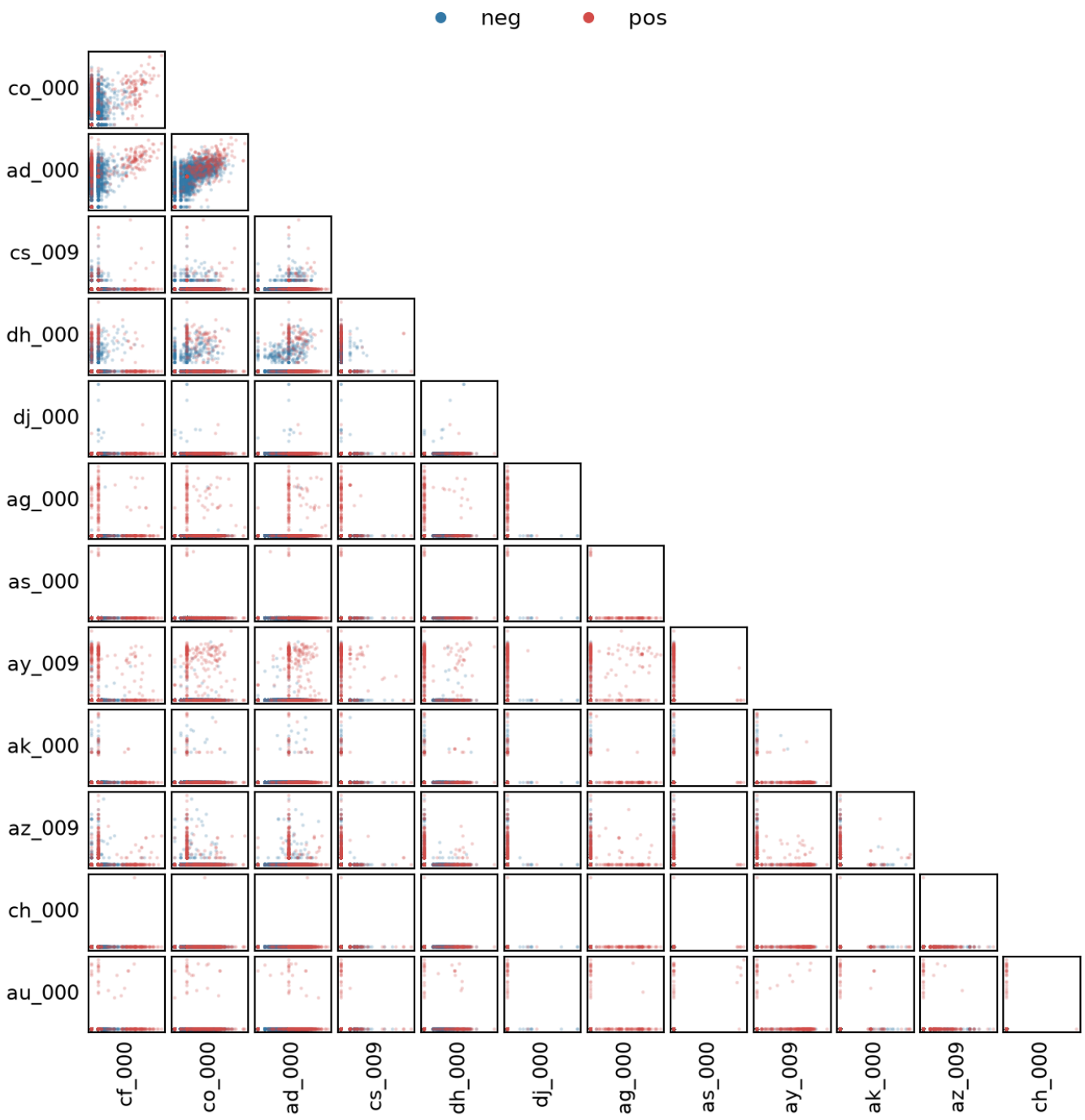


All 60,000 imputed training rows and all 170 features enter Pearson correlation. Plot indices are 1-based; code indices are zero-based. Correlation is marginal/linear, does not establish causation and can change after imputation. Related counters form correlated blocks.

```
def full_scatter(Z, ys, columns, colors):
    fig, axes = plt.subplots(12, 12, figsize=(7.4, 7.4))
    pairs = []
    for i in range(1, 13):
        for j in range(12):
            ax = axes[i-1, j]
            if j >= i:
                ax.axis('off'); continue
            ax.scatter(Z[columns[j]], Z[columns[i]], c=colors, s=.35, alpha=.22)
            ax.set_xticks([]); ax.set_yticks([])
            if i == 12: ax.set_xlabel(columns[j], rotation=90, fontsize=9)
            if j == 0: ax.set_ylabel(columns[i], rotation=0, ha='right', fontsize=9)
            pairs.append((columns[j], columns[i]))
    from matplotlib.lines import Line2D
    handles = [Line2D([0], [0], marker='o', ls='', color=c, label=l)
               for c, l in [('#3378a6', 'neg'), ('#d34c49', 'pos')]]
    fig.legend(handles=handles, loc='upper center', ncol=2, frameon=False)
    assert len(set(pairs)) == 78
    fig.subplots_adjust(left=.15, right=.99, bottom=.12, top=.95,
                        wspace=.08, hspace=.08)
    return fig
```

1(b)(iv). Scatter plots: the 13 highest-CV features

```
fig = full_scatter(Z, ys, top, colors)
# The report plots only j < i: 13*12/2 = 78 unordered pairs.
```

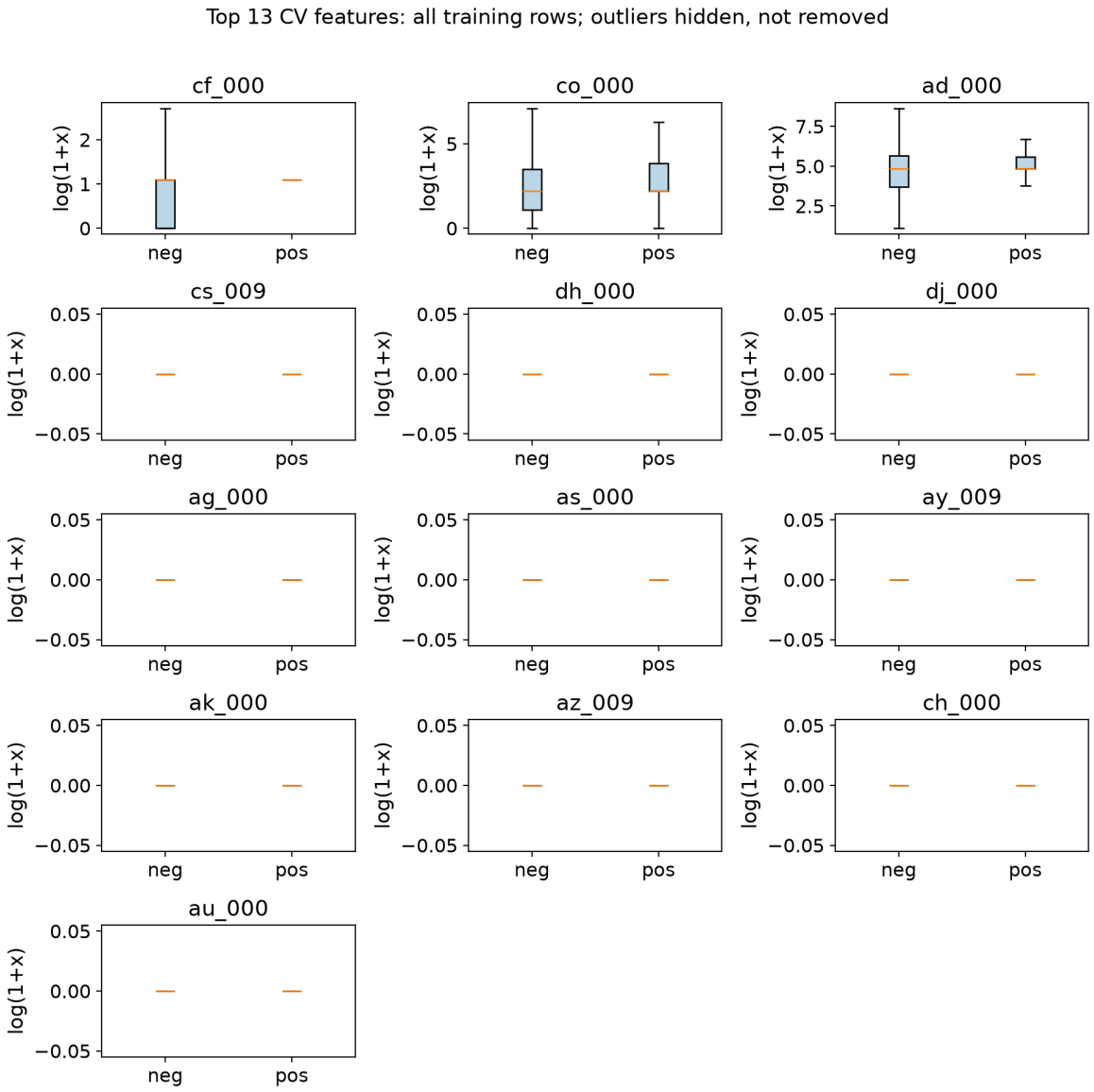


The lower triangle directly displays all 78 unordered pairs of the 13 selected features, with each pair appearing once. Blue = neg; red = pos. The plot uses all 1,000 positives plus 2,500 randomly sampled negatives, seed 552. This deliberate display sample does not represent prevalence. log(1+x) is used only for visualization, never model fitting.

Each panel uses its own log1p data range; numerical ticks are suppressed to keep feature names clear. The notebook also retains the full high-resolution matrix with ticks and diagonal histograms. Scatter plots alone cannot establish statistical significance: overplotting, marginal versus conditional effects, and multiple comparisons prevent an inferential conclusion. High CV is not predictive importance.

1(b)(iv)-(v). Box plots and imbalance

```
fig, axes = plt.subplots(5, 3, figsize=(10, 10))
for ax, col in zip(axes.flat, top):
    groups = [np.log1p(D.loc[y == c, col]) for c in [0, 1]]
    ax.boxplot(groups, tick_labels=['neg', 'pos'], showfliers=False)
    ax.set(title=col, ylabel='log(1+x)')
for ax in list(axes.flat)[13:]: ax.axis('off')
fig.tight_layout()
```



Boxes use all original training rows, separated by class, after training-median imputation and log(1+x) for display. Outlier symbols are hidden; no rows are removed. Several rare-count features have quartiles at zero, so boxes collapse.

Training: 59,000 neg and 1,000 pos (59:1; 1.667% positive). Official test: 15,625 neg and 375 pos (2.344% positive). Both are severely imbalanced. Always-negative prediction has error 1.667%/2.344% but zero positive recall. The 13 plotted predictors do not restrict modeling: all 170 are retained.

1(c)-(d). Common evaluation and forest training code

```
def evaluate(labels, probability):
    pred = (probability >= .5).astype(int)
    return dict(error=float(np.mean(pred != labels)),
        auc=M.roc_auc_score(labels, probability),
        average_precision=M.average_precision_score(labels, probability),
        precision=M.precision_score(labels, pred, zero_division=0),
        recall=M.recall_score(labels, pred), f1=M.f1_score(labels, pred),
        confusion_matrix=M.confusion_matrix(labels, pred, labels=[0, 1]))

def evaluate_splits(model, train, test, test_train_only):
    return {'train': evaluate(y, model.predict_proba(train)[:, 1]),
            'test': evaluate(yt, model.predict_proba(test)[:, 1]),
            'train_only_test': evaluate(yt,
                model.predict_proba(test_train_only)[:, 1])}

def covered_oob(model, labels):
    votes = np.zeros(len(labels), dtype=int)
    for bag in model.estimators_samples_:
        votes += np.bincount(bag, minlength=len(labels)) == 0
    p = model.oob_decision_function_
    valid = (votes > 0) & np.isfinite(p).all(1) & np.isclose(p.sum(1), 1)
    error = np.mean((p[valid, 1] >= .5) != labels[valid])
    return votes, valid, error  # Check coverage of EACH class before comparison.

def train_forests():
    fitted, scores = {}, {}
    for name, weight in [('rf_plain', None), ('rf_balanced', 'balanced')]:
        model = RandomForestClassifier(n_estimators=500, criterion='gini',
            max_features='sqrt', max_depth=None, min_samples_leaf=1,
            bootstrap=True, oob_score=True, class_weight=weight,
            random_state=SEED, n_jobs=4)
        model.fit(A, y)
        scores[name] = evaluate_splits(model, A, B, Bt)
        oob = model.oob_decision_function_[:, 1]
        scores[name]['historical_unmasked_oob'] = np.mean((oob >= .5) != y)
        scores[name]['oob_votes_mask_error'] = covered_oob(model, y)
        fitted[name] = model
    return fitted, scores
if RUN_TRAINING:
    forests, forest_scores = train_forests()

def draw_evaluation(labels, probability, cm_axis, roc_axis):
    cm = M.confusion_matrix(labels, probability >= .5, labels=[0, 1])
    M.ConfusionMatrixDisplay(cm, display_labels=['neg', 'pos']).plot(ax=cm_axis)
    fpr, tpr, _ = M.roc_curve(labels, probability)
    roc_axis.plot(fpr, tpr)

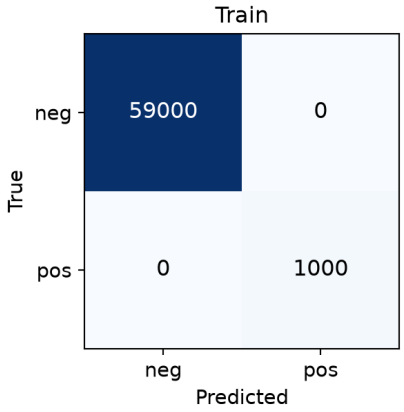
if RUN_TRAINING:
    supplement = RandomForestClassifier(**forests['rf_plain'].get_params())
    supplement.set_params(class_weight='balanced_subsample').fit(A, y)
    supplement_scores = evaluate_splits(supplement, A, B, Bt)
    supplement_oob = covered_oob(supplement, y)
```

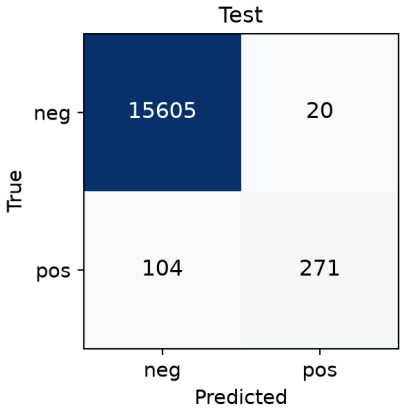
Imputation methods include deletion, constant/mean/median/mode, nearest neighbors, iterative models and multiple imputation [2]. Median filling resists extremes but distorts dependencies. Separate split medians follow the footnote; Bt is train-only sensitivity, and CV fits only tr. The plain forest has no class weighting. In sklearn 1.9.1, balanced weights n/(2 n_k)=0.5085/30 drive bootstrap sampling; balanced_subsample weights within uniformly drawn trees. Both are established imbalance strategies [3,4]; other parameters match.

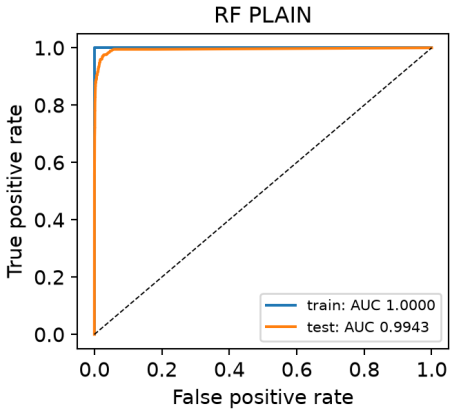
Predictions use P(pos)>=0.5, including ties. ROC-AUC uses probabilities; AP is average_precision_score. CM rows are true and columns predicted [neg,pos]. OOB averages only trees excluding that row; both-class coverage must be checked. RUN_TRAINING=False disables only PDF-layout fitting; the original full run and separate supplemental fit were actually executed.

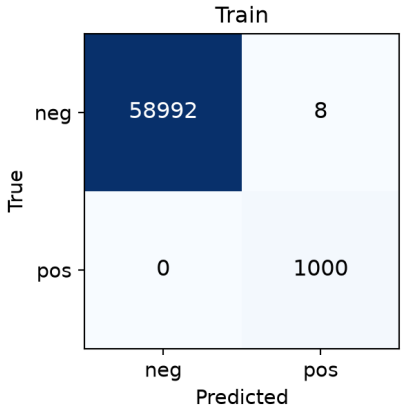
1(c)-(d). Forest outputs and OOB comparison

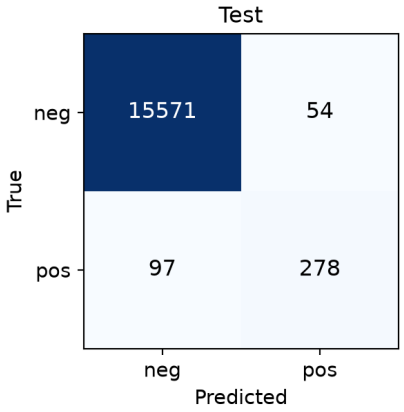
Model / split	Error	ROC-AUC	AP	Precision	Recall	F1
Plain / train	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000
Plain / test	0.007750	0.994347	0.926093	0.931271	0.722667	0.813814
Balanced / train	0.000133	1.000000	1.000000	0.992063	1.000000	0.996016
Balanced / test	0.009437	0.994177	0.887014	0.837349	0.741333	0.786421

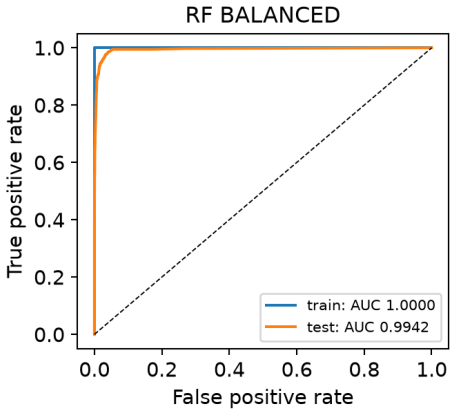












Forest	Historical all-row	Valid OOB error	Valid n	Full-class OOB?
Plain	0.00581667	0.00581667	60000	Yes
Balanced	0.01998333	0.00337288	59000	NO: negative-only

Plain: 349/60000 OOB errors; test error 0.00775000, gap +0.00193333. Balanced: all 1000 positives lack OOB votes and have [0,0] probabilities. The historical 1199/60000 counts them all as negatives/errors. Its valid-row value is 199/59000, NEGATIVE-ONLY; it cannot estimate overall error or be compared with full test error 0.00943750. Neither model has NaN or other invalid covered rows. Coverage/details and a separate valid supplement follow on pages 7-8.

Existing train/test scores above remain unchanged. Weighting slightly increases minority recall but lowers precision and increases overall error. Global training imputation makes otherwise valid OOB an approximate internal estimate; training scores are optimistic resubstitution results.

1(e)-(f). Pipeline; coverage-aware RF OOB review

XGBoost 3.4.1 gbtree uses feature-threshold splits and scalar leaf weights. binary:logistic is the logistic objective/link; reg_alpha penalizes leaf weights, not node-level logistic-regression coefficients [5,6]. gblinear is a global linear booster, not a model tree. Local course material gives no clarifying API mapping. The runnable standard-XGBoost interpretation is retained; a genuine logistic model tree remains an instructor question.

$$P(Y=1 \mid x)=\sigma(\sum_t f_t(x)), \quad \Omega(f_t)=\gamma L_t+\frac{\lambda}{2}\sum_\ell w_{t\ell}^2+\alpha\sum_\ell |w_{t\ell}|$$

```
ALPHAS = [0., .001, .01, .1, 1., 10., 100.]
def make_xgb(alpha):
    return XGBClassifier(booster='gbtree', objective='binary:logistic',
        reg_alpha=alpha, reg_lambda=1, n_estimators=250, max_depth=4,
        learning_rate=.08, subsample=1, colsample_bytree=1, min_child_weight=1,
        tree_method='hist', max_bin=256, base_score=.5, eval_metric='logloss',
        random_state=SEED, n_jobs=4)
def fold_pipeline(alpha, use_smote):
    steps = [('imputer', SimpleImputer(strategy='median',
                                       keep_empty_features=True)),
             ('scaler', StandardScaler())]
    if use_smote:
        steps += [('smote', SMOTE(random_state=SEED, k_neighbors=5,
                                  sampling_strategy=1.0))]
    return Pipeline(steps + [('estimator', make_xgb(alpha))])
```

The baseline has no imbalance compensation. Shared standardization makes SMOTE distances less dominated by large-unit counters; it cannot remove skew or high-dimensional distance limitations.

Pipeline.fit fits preprocessing and SMOTE only on tr; predict_proba transforms va without resampling. Test data never enter tuning. Rounds250/threshold0.5 and other settings are fixed; no early stopping or nested parallel CV.

RF OOB coverage audit and supplemental method

Forest / class	Zero votes	Valid / n	Coverage	Vote range / mean
Plain / neg	0	59000 / 59000	100.0%	144-228 / 184.00
Plain / pos	0	1000 / 1000	100.0%	151-214 / 184.31
Balanced / neg	0	59000 / 59000	100.0%	251-347 / 300.64
Balanced / pos	1000	0 / 1000	0.0%	0-0 / 0.00
Supplement / neg	0	59000 / 59000	100.0%	144-228 / 184.00
Supplement / pos	0	1000 / 1000	100.0%	151-214 / 184.31

Actual estimators_samples_ indices were counted for every tree and all stored OOB probabilities independently reconstructed from tree predictions. Overall valid coverage: plain 60000/60000; balanced 59000/60000 (98.3333%); supplement 60000/60000. In sklearn 1.9.1, balanced sampling gives each positive probability 1/2000 per draw: 30 expected draws/tree and near-zero exclusion probability. The generic too-few-trees warning comes from balanced, not plain. Dropping missing positives does not solve the coverage problem.

Split	Error	ROC-AUC	AP	Precision	Recall	F1
train	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000
test	0.010625	0.994543	0.886332	0.939914	0.584000	0.720395

The separate balanced_subsample forest preserves all other settings, uses uniform bootstraps plus per-tree balanced impurity weights, and covers both classes. OOB=457/60000=0.00761667; test error=0.01062500; gap=+0.00300833. It is not a replacement or test-selected improvement. Confusion matrices and ROC follow on page 8.

1(e)-(f). CV outputs; supplemental forest evaluation

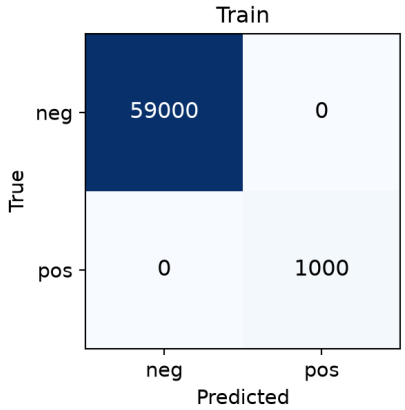
```
def search_alpha(use_smote):
    folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
    rows = []
    for alpha in ALPHAS:
        for fold, (tr, va) in enumerate(folds.split(X, y), 1):
            pipe = fold_pipeline(alpha, use_smote)
            pipe.fit(X.iloc[tr], y[tr])  # Only this training fold is fitted.
            p = pipe.predict_proba(X.iloc[va])[:, 1]  # Never resampled.
            rows.append(dict(alpha=alpha, fold=fold,
                ap=M.average_precision_score(y[va], p),
                error=float(np.mean((p >= .5) != y[va])),
                auc=M.roc_auc_score(y[va], p)))
    scores = pd.DataFrame(rows)
    summary = scores.groupby('alpha').agg(mean_ap=('ap', 'mean'),
        sd_ap=('ap', 'std'), mean_error=('error', 'mean'),
        sd_error=('error', 'std'), mean_auc=('auc', 'mean')).reset_index()
    best = summary.sort_values(['mean_ap', 'alpha'],
                               ascending=[False, True]).iloc[0]['alpha']
    return scores, summary, float(best)
if RUN_TRAINING:
    plain_folds, plain_summary, alpha0 = search_alpha(False)
    smote_folds, smote_summary, alpha1 = search_alpha(True)
```

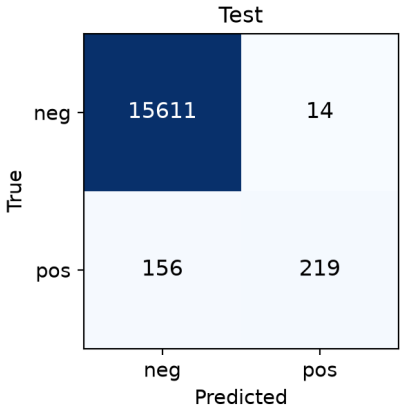
Alpha	Plain mean AP	Plain CV error	SMOTE mean AP	SMOTE CV error
0	0.887600	0.005617	0.847497	0.009767
0.001	0.885676	0.005767	0.847613	0.009750
0.01	0.887273	0.005683	0.849106	0.009850
0.1	0.886433	0.005817	0.848896	0.009900
1	0.884724	0.005983	0.849738	0.009900
10	0.869320	0.006583	0.841769	0.010517
100	0.783458	0.009433	0.786648	0.020167

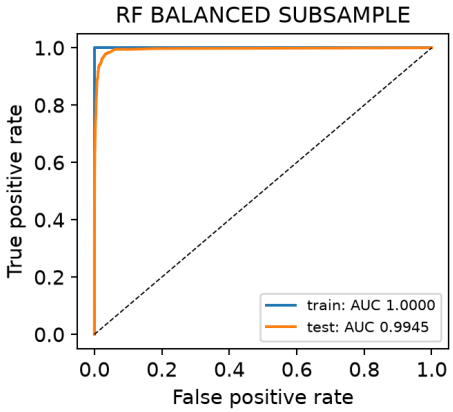
Both searches use all training rows, identical stratified five-fold splits and fold-local preprocessing. Select mean AP; smaller-alpha ties. No test-based selection.

Selected plain alpha=0 (no L1 penalty), SMOTE alpha=1. Earlier full-run CV times: 171.2/287.4s. Selected CV scores are tuning estimates, not unbiased nested validation.

Separate balanced_subsample forest: actual outputs



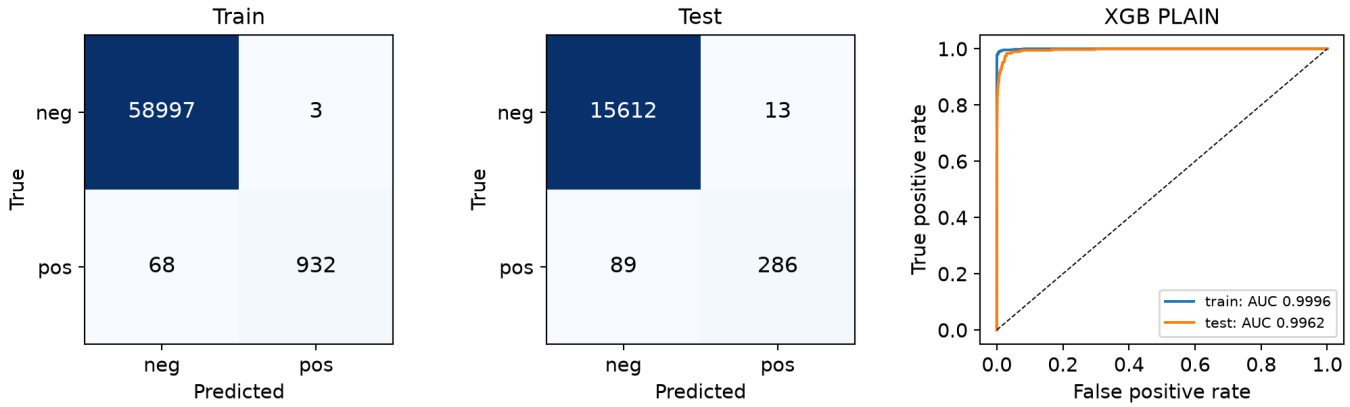




1(e). Full training and uncompensated XGBoost results

```
def train_final(alpha, use_smote):
    import time
    scaler = StandardScaler()
    As = scaler.fit_transform(A)
    Bs, Bts = scaler.transform(B), scaler.transform(Bt)
    start = time.perf_counter()
    if use_smote:
        Ar, yr = SMOTE(random_state=SEED, k_neighbors=5,
                       sampling_strategy=1.0).fit_resample(As, y)
    else:
        Ar, yr = As, y
    sampling_seconds = time.perf_counter() - start
    model = make_xgb(alpha)
    model.fit(Ar, yr)
    # Evaluation always uses original rows, including for the training scores.
    scores = evaluate_splits(model, As, Bs, Bts)
    counts = np.bincount(yr, minlength=2)
    return model, scaler, scores, counts, sampling_seconds
if RUN_TRAINING:
    plain_final = train_final(alpha0, False)
```

Split	Error	ROC-AUC	AP	Precision	Recall	F1
train	0.001183	0.999593	0.990880	0.996791	0.932000	0.963307
test	0.006375	0.996230	0.931994	0.956522	0.762667	0.848665



No compensation; alpha=0. Test minus CV error = +0.000758. CV uses 80% fitting rows and fold-local preprocessing; the final fit uses all rows and the test-imputation footnote. Current full-run final fit/evaluation: 4.53s. Training scores use original rows.

Supplementary comparison restricted to alpha > 0

```
def positive_alpha(summary):
    positive = summary.loc[summary.alpha > 0]
    return float(positive.sort_values(['mean_ap', 'alpha'],
                                      ascending=[False, True]).iloc[0].alpha)
if RUN_TRAINING:
    positive_final = train_final(positive_alpha(plain_summary), False)
```

Split	alpha	CV error	Error	ROC-AUC	CM: [[TN,FP],[FN,TP]]
train	0.01	0.005683	0.001133	0.999726	[[58997, 3], [65, 935]]
test	0.01	0.005683	0.006125	0.996063	[[15612, 13], [85, 290]]

Restrict the same CV AP table to alpha > 0; the same smaller-alpha tie rule selects 0.01 (CV AP 0.887273). A separate all-60,000-row, no-SMOTE model was actually fitted with every other setting fixed. Test error difference versus alpha=0: -0.000250; AUC difference: -0.000167. It does not replace the unrestricted optimum or resolve the node-level model-tree ambiguity.

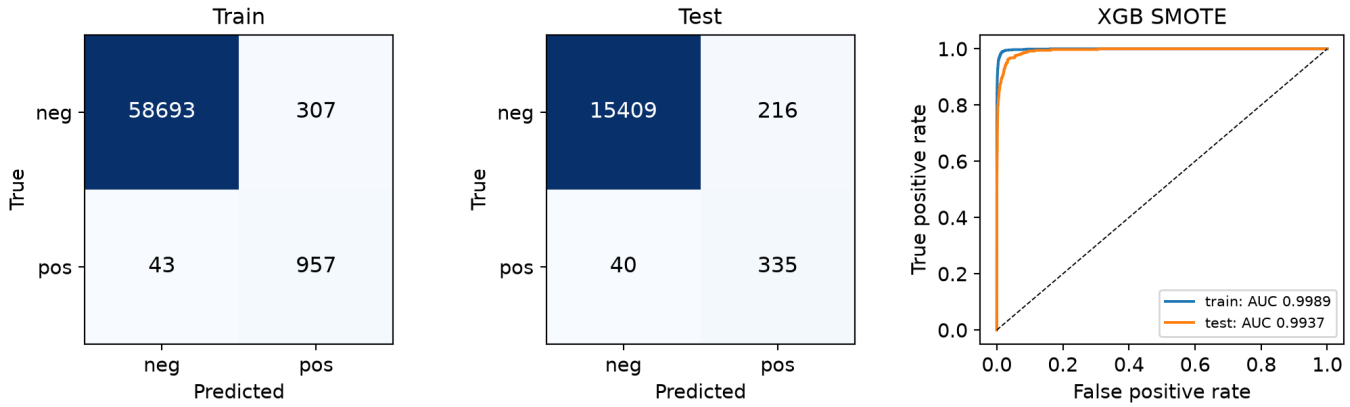
1(f). Full-data SMOTE + XGBoost results

```
if RUN_TRAINING:
    smote_final = train_final(alpha1, True)
    print(smote_final[3])  # Final class counts: [59000, 59000].
```

$$x_{new} = x_i + u(x_{neighbor} - x_i), \qquad u \sim U(0,1)$$

Stage	neg	Original pos	Final pos	Total
Each fold fit	47200	800	47200	94400
Full final fit	59000	1000	59000	118000
Validation (unchanged)	11800	200	200	12000
Test (unchanged)	15625	375	375	16000

Split	Error	ROC-AUC	AP	Precision	Recall	F1
train	0.005833	0.998855	0.963421	0.757120	0.957000	0.845406
test	0.016000	0.993653	0.883624	0.607985	0.893333	0.723542



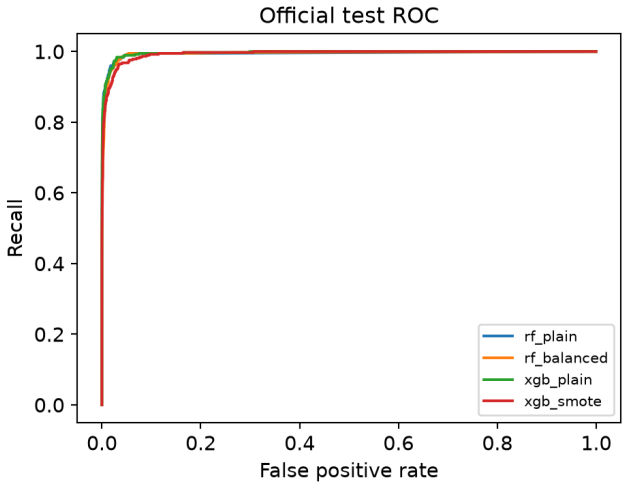
SMOTE [7] uses five minority neighbors, seed 552, sampling_strategy=1.0, after fold-local imputation and standardization. There is no majority downsampling. Final full-data resampling took 0.155s; final scaling, resampling, fit/evaluation took 7.26s. All full-data and fold fits were executed anew in the isolated full run. Original training/test distributions are used for all displayed evaluations.

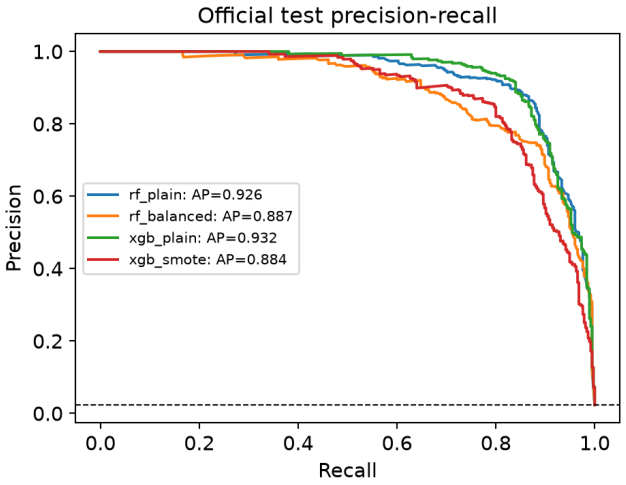
Alpha = 1; test error 0.016000 minus selected CV error 0.009900 = 0.006100. Compared with plain XGB, recall changes 0.7627 to 0.8933, precision 0.9565 to 0.6080; FP 13 to 216, FN 89 to 40. Oversampling changes the fitted prior, so scores need not be calibrated population probabilities. The threshold was not optimized on the test set. Resampling before splitting would leak neighbor information [8].

1(c)-(f). Unified comparison and imputation sensitivity

```
def pr_curves(predictions):
    fig, ax = plt.subplots(figsize=(6, 4))
    for name, p in predictions.items():
        precision, recall, _ = M.precision_recall_curve(yt, p)
        ap = M.average_precision_score(yt, p)
        ax.plot(recall, precision, label=f'{name}: AP={ap:.3f}')
    ax.axhline(yt.mean(), ls='--', color='black')
    ax.set(xlabel='Recall', ylabel='Precision')
    ax.legend()
    return fig
# Bt (p.1) and evaluate_splits (p.5) produce the sensitivity evaluation.
```

Model / test	Error	ROC-AUC	AP	Precision	Recall	F1
RF plain	0.00775	0.99435	0.92609	0.93127	0.72267	0.81381
RF balanced	0.00944	0.99418	0.88701	0.83735	0.74133	0.78642
XGB plain	0.00637	0.99623	0.93199	0.95652	0.76267	0.84866
XGB SMOTE	0.01600	0.99365	0.88362	0.60799	0.89333	0.72354





Model	Main error	Train-only error	Main AP	Train-only AP
RF plain	0.007750	0.007750	0.92609	0.92614
RF balanced	0.009437	0.009437	0.88701	0.88716
XGB plain	0.006375	0.006250	0.93199	0.93160
XGB SMOTE	0.016000	0.010750	0.88362	0.89974

Sensitivity holds every model and scaler fixed, replacing only test-median B with train-median Bt. No result in this table selects a model or threshold. SMOTE main versus train-only error: 0.016000 versus 0.010750; FP 216 versus 125, FN 40 versus 47. This material change shows how synthetic-neighbor fits can amplify shifts in imputed values.

ROC-AUC measures ranking across both classes; AP summarizes precision along minority recall and depends on prevalence. The dashed PR baseline is test prevalence 0.02344. The data description [9] gives cost 10 FP + 500 FN. Retrospective test costs (RF plain/balanced, XGB plain/SMOTE): 52200, 49040, 44630, 22160. Costs did not guide tuning.

Q2 and Q6 ridge: all regularization subparts

Q2: ISLR 2nd ed. 6.6.3 (a)-(e), p.284; same content as 1st ed. 6.8.3. Q6 Extra Practice: exact assignment reference ISLR 1st ed. 6.8.4 (a)-(e), pp.260-261; its 2nd-edition equivalent is 6.6.4 (not 6.8.4), pp.284-285. These are distinct lasso-radius and ridge-penalty questions. Intercepts are unpenalized.

$$Q2:\ \min R(\beta)\ \text{s.t.}\ \sum_j |\beta_j| \le s; \qquad Q6:\ \min R(\beta) + \lambda \sum_j \beta_j^2$$

Subpart	Q2: increase lasso radius s	Q6: increase ridge penalty lambda
(a) Training RSS	iv: nonincreasing	iii: nondecreasing
(b) Test RSS	ii: typical U shape	ii: typical U shape
(c) Variance	iii: typically increases	iv: decreases
(d) Squared bias	iv: typically decreases	iii: typically increases
(e) Irreducible error	v: constant	v: constant

Q2 explanations (a)-(e)

(a) Larger s enlarges the feasible set: the previous minimizer remains feasible, so training RSS cannot increase. It may plateau once least squares is feasible. At s=0, slopes are zero and only the sample-mean intercept remains. (b) Relaxing a tight constraint first reduces underfitting; excessive flexibility can increase variance and test error. (c) Less shrinkage permits greater sensitivity to sample noise. (d) Reduced shrinkage generally removes bias. (e) The response noise variance belongs to the generating process, not the fitting method.

Q6 ridge explanations (a)-(e)

(a) Let R1,P1 and R2,P2 be RSS and squared-coefficient penalty values at minimizers for 0 <= lambda1 < lambda2. Optimality gives R1+lambda1 P1 <= R2+lambda1 P2 and R2+lambda2 P2 <= R1+lambda2 P1. Adding implies P2 <= P1; substituting yields R2-R1 >= lambda1(P1-P2) >= 0. Equality is possible. (b) Moderate shrinkage can reduce variance enough to improve prediction; too much shrinkage underfits. (c) For centered fixed design X=UDV-transpose and homoskedastic noise, each identifiable singular-direction variance decreases as below. The intercept contributes a constant term.

$$\text{Var}(\hat{\beta}_{\lambda,k}) = \frac{\sigma^2 d_k^2}{(d_k^2+\lambda)^2}, \qquad \text{Bias}_k = -\frac{\lambda}{d_k^2+\lambda}\beta_k$$

(d) Increasing lambda shrinks slopes farther toward zero, generally increasing squared bias. This is monotone in the orthogonal coefficient norm (or isotropic test average), but arbitrary pointwise prediction biases can cancel across directions. (e) Regularization cannot change irreducible response noise. The U-shaped test errors and lasso bias/variance trends are intended statistical patterns, not universal monotonic theorems for every realized sample. Only the nested-set/optimality training-RSS statements are guaranteed. No numerical computation is required by either exercise; the notebook retains optional illustrative plots.

Q3. ISLR 6.6.5: identical predictors and solutions

ISLR second edition Section 6.6 Exercise 5 (first edition: 6.8.5). Let x1 = x11 = x12 and x2 = x21 = x22, with centered x and y, so the fitted intercept is zero. Assume a nonzero predictor column and a positive penalty unless stated otherwise. Put t = beta1 + beta2, A = x1 squared + x2 squared > 0, and C = x1 y1 + x2 y2.

(a) Ridge objective

$$\min_{\beta_1,\beta_2} (y_1 - x_1\beta_1 - x_1\beta_2)^2 + (y_2 - x_2\beta_1 - x_2\beta_2)^2 + \lambda(\beta_1^2 + \beta_2^2)$$

(b) Why the ridge coefficients are equal

For a fixed sum t, the residual term depends only on t. The identity below shows that the penalty is uniquely minimized by beta1 = beta2 = t/2. Equivalently, subtracting the two normal equations gives 2 lambda (beta1 - beta2) = 0. Strict convexity from lambda > 0 makes the solution unique.

$$\beta_1^2 + \beta_2^2 = \frac{t^2}{2} + \frac{(\beta_1 - \beta_2)^2}{2}, \qquad \hat{\beta}_1 = \hat{\beta}_2 = \frac{C}{2A + \lambda}$$

(c) Lasso objective

$$\min_{\beta_1,\beta_2} (y_1 - x_1\beta_1 - x_1\beta_2)^2 + (y_2 - x_2\beta_1 - x_2\beta_2)^2 + \lambda(|\beta_1| + |\beta_2|)$$

(d) Describe all minimizing coefficient pairs

The triangle inequality gives |beta1| + |beta2| >= |t|, with equality whenever the two coefficients have the same sign (allowing zero). Thus minimizing reduces to the one-dimensional objective A t squared - 2 C t + lambda |t| plus a constant. Its minimizer is the soft-thresholded sum below.

$$t^* = \frac{\text{sign}(C)(|C| - \lambda/2)_+}{A}, \qquad (\hat{\beta}_1, \hat{\beta}_2) = (at^*, (1-a)t^*), \quad 0 \le a \le 1$$

When t* is nonzero, this continuum includes selecting only the first predictor, selecting only the second, and every same-sign split between them. All pairs have identical fitted values and identical L1 penalty. This explains why a lasso solver can treat perfectly correlated variables differently.

Boundary qualifications: if lambda >= 2|C| and lambda > 0, t* = 0 and only (0,0) minimizes, so nonuniqueness is not universal. If lambda = 0, any pair summing to C/A is least-squares optimal, including opposite signs; the equality claim for ridge is then not forced. These qualifications refine the exercise's generic nonuniqueness statement.

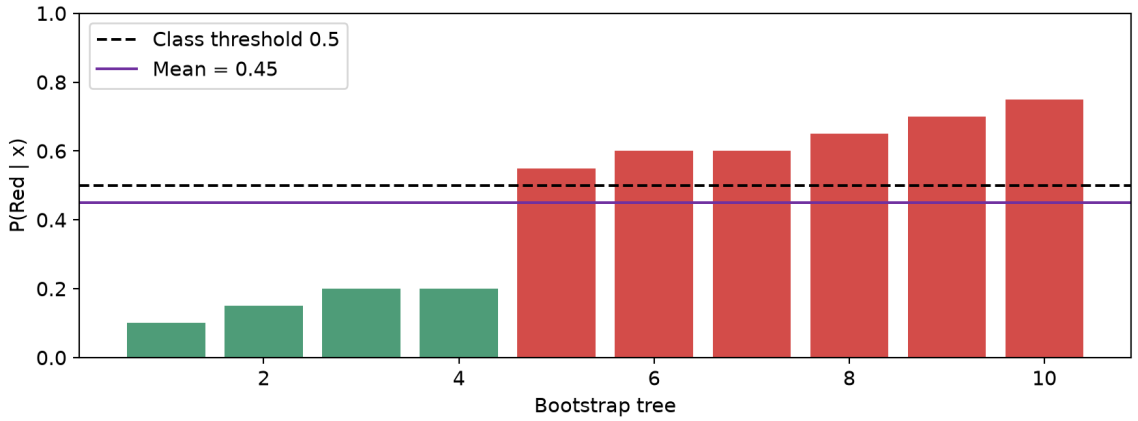
```
x = np.array([1., -1.]); response = np.array([2., -2.]); penalty = 2.
a = x @ x; c = x @ response
ridge = c / (2*a + penalty)
tstar = np.sign(c) * max(abs(c) - penalty/2, 0) / a
objectives = [np.sum((response - x*tstar)**2) + penalty *
              (abs(w*tstar) + abs((1-w)*tstar)) for w in np.linspace(0, 1, 11)]
assert np.ptp(objectives) < 1e-12
```

```
Centered check: ridge slopes = (0.6666667, 0.6666667)
Lasso optimal sum = 1.5; all 11 same-sign splits give objective 3.5.
```

Q4. ISLR 8.4.5: bootstrap votes vs mean probability

Both checked R editions identify this exercise as Section 8.4, Exercise 5. The ten red probabilities in the code are the given inputs.

```
prob = np.array([.1, .15, .2, .2, .55, .6, .6, .65, .7, .75])
red_votes = int((prob > .5).sum())
red_probability = float(prob.mean())
fig, ax = plt.subplots(figsize=(9, 3.5))
ax.bar(np.arange(1, 11), prob)
ax.axhline(.5, color='black', ls='--')
ax.axhline(red_probability, color='purple')
ax.set(xlabel='Bootstrap tree', ylabel='P(Red)', ylim=(0, 1))
```



Majority vote: the first four trees vote Green and the final six vote Red, so the result is Red (6 of 10 votes). No probability is exactly 0.5. Mean probability: summing all probabilities gives 4.5, hence mean P(Red)=0.45 and P(Green)=0.55; the probability-average rule predicts Green.

$$\overline{p}_{Red} = 4.5/10 = 0.45 < 0.5$$

Hard voting discards confidence, whereas averaging retains numerical magnitudes. Several low red probabilities can outweigh more numerous moderately high ones. The rules need not agree. The Question 1 sklearn forests average probabilities before thresholding, as shown in their common evaluator.

Shared plotting helper for Q5 (called on page 15)

```
def geometry_plot(points, labels):
    fig, ax = plt.subplots(figsize=(5, 5))
    for cls, color in [(1, '#d34c49'), (-1, '#3378a6')]:
        ax.scatter(*points[labels == cls].T, c=color)
    for i, point in enumerate(points, 1): ax.annotate(str(i), point)
    xx = np.linspace(.5, 4.8, 200)
    for offset, style in [(-.5, '-'), (0, '--'), (-1, '--')]:
        ax.plot(xx, xx + offset, color='black', ls=style)
    ax.plot(xx, xx-.2, ':', color='purple')
    ax.scatter([3], [3], c='#3378a6', marker='X')  # Added Blue point for (h).
    ax.set(xlim=(.5, 4.8), ylim=(.3, 4.8), xlabel='X1', ylabel='X2', aspect='equal')
    return fig
```

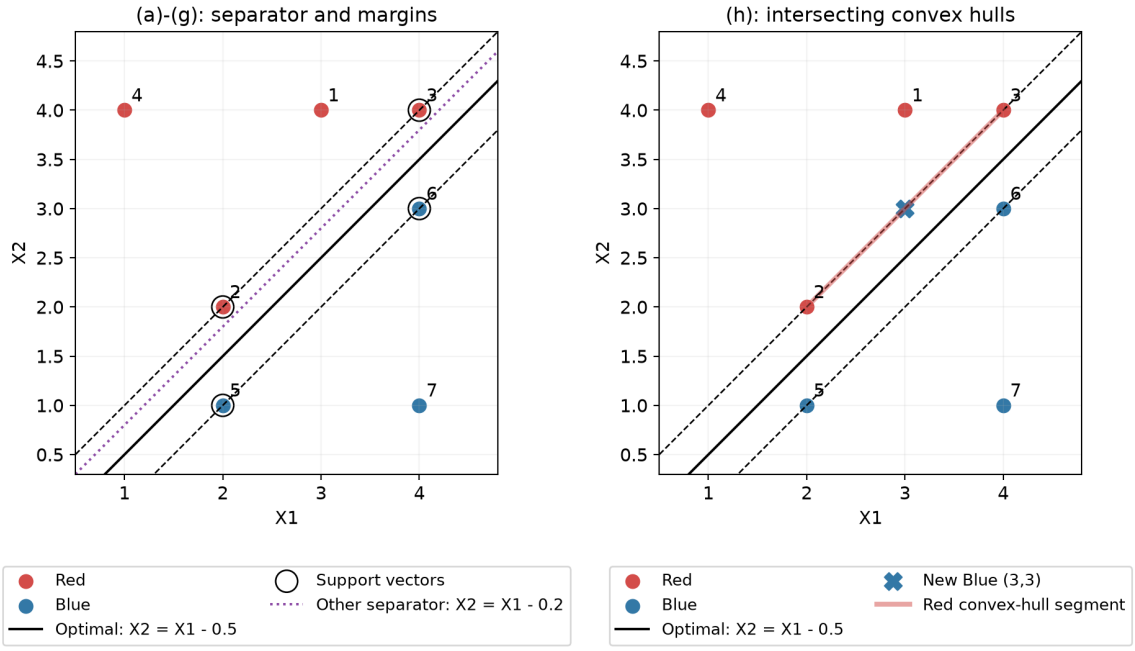
Q5. ISLR 9.7.3: maximal-margin geometry (a)-(h)

Both checked R editions use Exercise 9.7.3. (a) The numbered coordinates are given in the executed code and figure. Red is +1, Blue is -1; the hard-margin approximation is an independent numerical verification, not an APS model.

```
from sklearn.svm import SVC
points = np.array([[3,4], [2,2], [4,4], [1,4], [2,1], [4,3], [4,1]])
labels = np.array([1, 1, 1, 1, -1, -1, -1])
svm = SVC(C=1e6, kernel='linear', tol=1e-10).fit(points, labels)
assert set(svm.support_ + 1) == {2, 3, 5, 6}
margin = 1 / np.linalg.norm(svm.coef_[0])

geometry_plot(points, labels)
```

```
SVM: coefficients (-2, 2); intercept 1; support observations {2,3,5,6}.
Geometric margin = 0.3535533906.
```



(b) The optimal boundary is X2 = X1 - 0.5, or 0.5 - X1 + X2 = 0. The nearest Red and Blue points lie on X2 = X1 and X2 = X1 - 1. Their convex hulls are 1/sqrt(2) apart; the midpoint line attains half this distance, the largest possible margin.

(c) Classify Red if 0.5 - X1 + X2 > 0; otherwise classify Blue. Thus (beta0, beta1, beta2) = (0.5, -1, 1); a positive rescaling gives the same classifier.

(d) The geometric margin on each side is 0.5/sqrt(2) = 0.353553. The full width between supporting lines is 1/sqrt(2) = 0.707107. In canonical functional-margin units use (1, -2, 2), with support scores +1 or -1.

(e) Support vectors are observations 2, 3, 5 and 6, on the dashed supporting lines. An independent hard-margin SVM approximation verifies these points, coefficients (-2,2), and intercept 1.

(f) Observation 7 has score 0.5 - 4 + 1 = -2.5. A sufficiently small movement keeps it strictly beyond the Blue supporting line, leaving the active margin constraints and optimal boundary unchanged.

(g) The purple dotted separator X2 = X1 - 0.2 still separates every original point. Its closest distance is 0.2/sqrt(2), smaller than 0.5/sqrt(2), so it is not optimal.

(h) Add a Blue point at (3,3), the midpoint of Red observations 2=(2,2) and 3=(4,4). A linear functional positive at both Red endpoints is positive at their midpoint. The convex hulls intersect, so no hyperplane can strictly separate the classes.

Q6 Extra Practice. ISLR 5.4.2: bootstrap inclusion

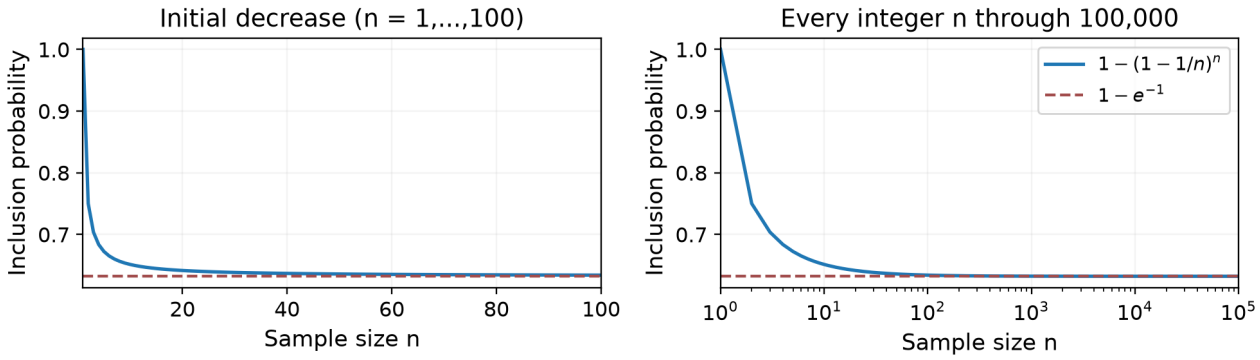
Individually verified: 1st ed. 5.4.2, pp.197-198 (PDF 212-213); 2nd ed. 5.4.2, pp.219-220, has the same (a)-(h). Bootstrap draws are independent, uniform and with replacement.

```
n = np.arange(1, 100001)
included = np.ones(len(n))
included[1:] = -np.expm1(n[1:] * np.log1p(-1/n[1:]))
rng = np.random.default_rng(SEED)
samples = rng.integers(1, 101, size=(10000, 100))
hits = np.any(samples == 4, axis=1)
empirical = float(hits.mean())
exact = float(included[99])
mcse = np.sqrt(exact * (1-exact) / 10000)
fig, ax = plt.subplots(figsize=(8, 2.8))
ax.plot(n, included); ax.set_xscale('log')
ax.axhline(1-np.exp(-1), color='red', ls='--')
ax.set(xlabel='n (all integers 1 through 100000)', ylabel='Inclusion probability')
```

(a) The first draw misses fixed observation j with probability (n-1)/n=1-1/n, since n-1 equally likely indices differ from j. (b) Replacement restores the same population, so the second draw has the same miss probability. (c) Independence multiplies the n miss probabilities; inclusion is the complement.

$$P(j \notin B) = (1 - 1/n)^n, \quad P(j \in B) = 1 - (1 - 1/n)^n$$

Part	n	Exact inclusion probability
(d)	5	0.6723200000
(e)	100	0.6339676587
(f)	10000	0.6321389536



(g) Every integer n=1,...,100000 is evaluated; the right display uses a log axis. Inclusion decreases from 1 toward 1-exp(-1)=0.6321205588 from above. About 36.8% of original observations are absent even though there are n draws, because indices repeat. (h) The actual seeded 10,000-sample run includes j=4 in 6238 samples: 0.6238, versus exact 0.633968. Monte Carlo SE is 0.00482; the difference is -2.11 SE. The realization need not equal R's generator.

Q6 Extra Practice. ISLR 8.4.4: executable tree/region code

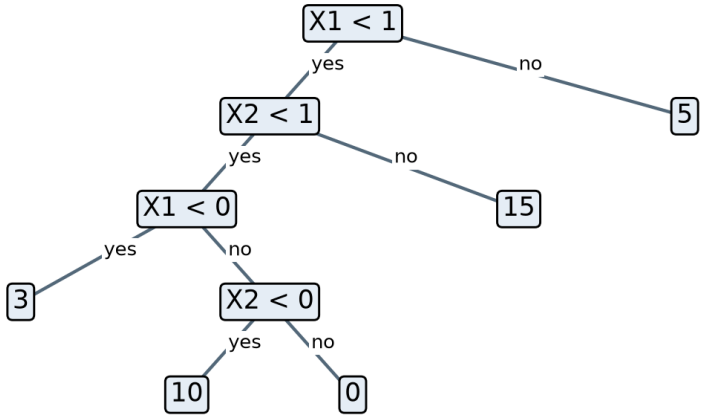
Verified separately in 1st ed. 8.4.4, pp.332-333, Figure 8.12, and 2nd ed. 8.4.4, pp.361-362, Figure 8.14: all cuts and means agree. Each nested tuple is (zero-based feature index, cut, true branch, false branch); a scalar is a leaf mean. RA/RB store (left, bottom, width, height, mean) for a finite display window. Outer regions extend beyond the window.

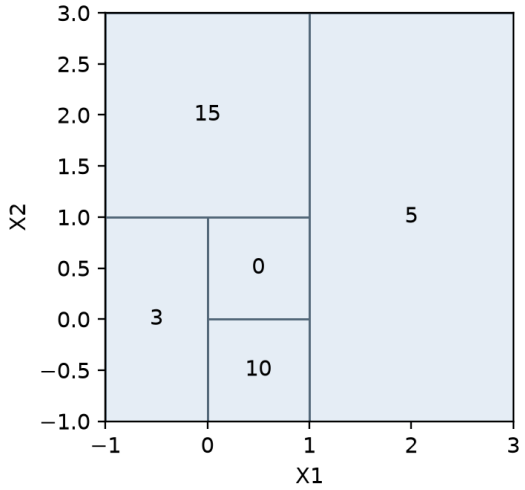
```
TA = (0, 1, (1, 1, (0, 0, 3, (1, 0, 10, 0)), 15), 5)
TB = (1, 1, (0, 1, -1.80, .63), (1, 2, (0, 0, -1.06, .21), 2.49))
def leaf(t): return not isinstance(t, tuple)
def leaves(t): return 1 if leaf(t) else leaves(t[2]) + leaves(t[3])
def tree_value(t, x):
    if leaf(t): return t
    j, cut, yes, no = t
    return tree_value(yes if x[j] < cut else no, x)
def draw_tree(ax, t, lo=0, hi=1, depth=0):
    x, y0 = (lo+hi)/2, -depth
    if leaf(t):
        text = f'{t:g}'
    else:
        j, cut, yes, no = t
        text = f'X{j+1} < {cut:g}'
        mid = lo + (hi-lo)*leaves(yes)/leaves(t)
        for branch, a, b, label in [(yes,lo,mid,'yes'), (no,mid,hi,'no')]:
            cx, cy = draw_tree(ax, branch, a, b, depth+1)
            ax.plot([x,cx], [y0,cy], color='#536879', zorder=0)
            ax.text((x+cx)/2, (y0+cy)/2, label, fontsize=9,
                    bbox=dict(facecolor='white', edgecolor='none', pad=1))
    ax.text(x, y0, text, ha='center', va='center', fontsize=12,
            bbox=dict(boxstyle='round,pad=.2', facecolor='#e5edf5'))
    return x, y0
RA = [(-1,-1,1,2,3), (0,-1,1,1,10), (0,0,1,1,0),
      (-1,1,2,2,15), (1,-1,2,4,5)]
RB = [(-1,-1,2,2,-1.80), (1,-1,2,2,.63), (-1,1,1,1,-1.06),
      (0,1,3,1,.21), (-1,2,4,1,2.49)]
def tree_partition(t, regions):
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.4))
    draw_tree(axes[0], t); axes[0].axis('off')
    for x, y0, w, h, mean in regions:
        axes[1].add_patch(plt.Rectangle((x,y0), w, h, facecolor='#e5edf5',
                                        edgecolor='#536879'))
        axes[1].text(x+w/2, y0+h/2, f'{mean:g}', ha='center', va='center')
    axes[1].set(xlim=(-1,3), ylim=(-1,3), xlabel='X1', ylabel='X2', aspect='equal')
    fig.tight_layout()
    return fig
fig_a = tree_partition(TA, RA)
fig_b = tree_partition(TB, RB)
```

The code is executable plotting and traversal logic, not pseudocode. It produces the complete corresponding tree for (a), and the requested partition for (b), alongside both source structures redrawn for comparison. Equality follows the false branch of each strict less-than split. Outputs and geometric explanations follow on page 18.

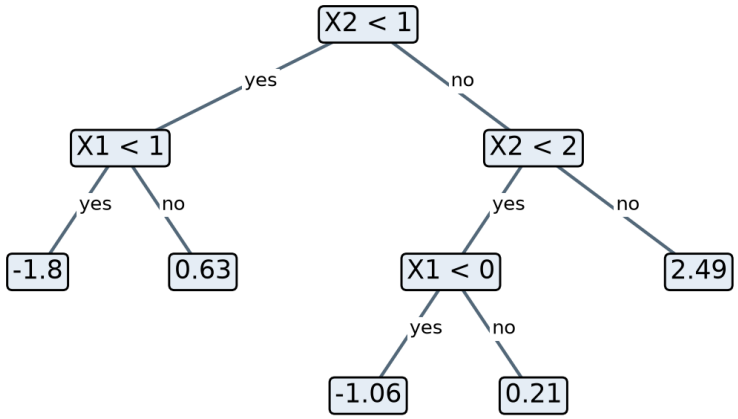
Q6 Extra Practice. ISLR 8.4.4: diagrams and answers

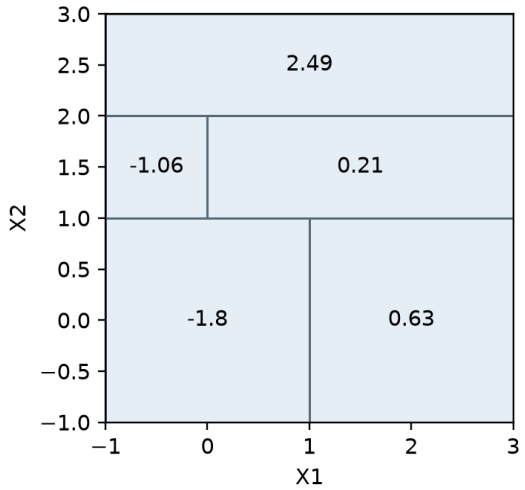
(a) Root X1 < 1: false gives 5. On the true branch, split X2 < 1: false gives 15. In the lower-left branch split X1 < 0: true gives 3; otherwise split X2 < 0, giving 10 if true and 0 if false. This valid recursive tree exactly matches the source partition.





(b) Root X2 < 1: in the lower half X1 < 1 gives -1.80, otherwise 0.63. In the upper half X2 < 2 selects the middle strip, where X1 < 0 gives -1.06 and otherwise 0.21. The uppermost region X2 >= 2 has mean 2.49.





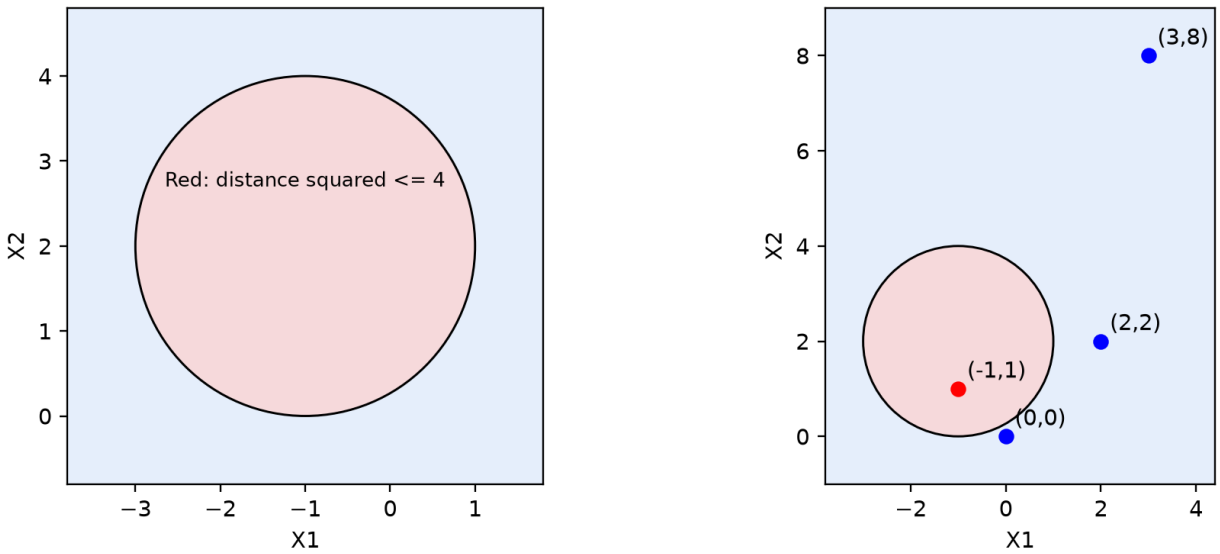
Region conditions	Mean Y
X2 < 1, X1 < 1	-1.80
X2 < 1, X1 >= 1	0.63
1 <= X2 < 2, X1 < 0	-1.06
1 <= X2 < 2, X1 >= 0	0.21
X2 >= 2	2.49

Independent recursive traversal and the original piecewise formulas agree on all 6,561 grid points, including cut lines. Each tree and partition has five leaves/regions. No fitted statistical model is needed; these are direct constructions from the supplied figure.

Q6 Extra Practice. ISLR 9.7.2: nonlinear boundary

Verified separately: 1st ed. 9.7.2, p.368; 2nd ed. 9.7.2, p.398, has the same (a)-(d). (a) The circle has center (-1,2), radius 2, horizontal extremes (-3,2)/(1,2) and vertical extremes (-1,0)/(-1,4). (b) >4 denotes the exterior; <=4 the closed disk. Exterior is Blue and the disk, including its boundary, is Red.

```
query = np.array([[0,0], [-1,1], [2,2], [3,8]])
distance2 = ((query - [-1,2])**2).sum(axis=1)
score = query[:,0]**2 + query[:,1]**2 + 2*query[:,0] - 4*query[:,1] + 1
classes = np.where(score > 0, 'Blue', 'Red')
assert np.array_equal(score, distance2 - 4)
fig, axes = plt.subplots(1, 2, figsize=(9.6, 3.7))
for ax in axes:
    ax.set_facecolor('#e5eefb')
    ax.add_patch(plt.Circle((-1,2), 2, facecolor='#f6d9db', edgecolor='black'))
    ax.set(xlabel='X1', ylabel='X2', aspect='equal')
axes[0].set(xlim=(-3.8,1.8), ylim=(-.8,4.8))
axes[0].text(-1, 2.7, 'Red: distance squared <= 4', ha='center', fontsize=9)
axes[1].set(xlim=(-3.8,4.4), ylim=(-1,9))
for point, cls in zip(query, classes):
    axes[1].scatter(*point, c='red' if cls == 'Red' else 'blue')
    axes[1].annotate(f'({point[0]},{point[1]})', point, xytext=(4,5),
                     textcoords='offset points', fontsize=10)
fig.tight_layout()
```



Part (c) point	Squared distance	Class
(0,0)	5	Blue
(-1,1)	1	Red
(2,2)	9	Blue
(3,8)	52	Blue

$$(1+X_1)^2+(2-X_2)^2-4=X_1^2+X_2^2+2X_1-4X_2+1$$

(d) In z=(X1, X1 squared, X2, X2 squared), the boundary is 1+2z1+z2-4z3+z4=0: an affine hyperplane with intercept 1 and coefficients (2,1,-4,1). Positive score predicts Blue; otherwise Red. The original-input boundary is nonlinear. Computed scores (1,-3,5,48) equal squared distance minus 4 exactly.

References, execution provenance and submission scope

[1] James, G., Witten, D., Hastie, T., and Tibshirani, R. An Introduction to Statistical Learning, with Applications in R, 2nd ed. (2021), corrected June 21, 2023; compared against first-edition seventh printing (2013). Author-provided downloads: https://www.statlearning.com/ . Second-edition download redirect: https://hastie.su.domains/ISLR2/ISLRv2_corrected_June_2023.pdf.download.html . Verified exercise locations and source-page images are retained locally.

[2] Scikit-learn, Imputation of missing values: https://scikit-learn.org/stable/modules/impute.html .

[3] Chen, C., Liaw, A., and Breiman, L. (2004). Using Random Forest to Learn Imbalanced Data. Technical Report 666, UC Berkeley. https://statistics.berkeley.edu/sites/default/files/tech-reports/666.pdf .

[4] Scikit-learn, RandomForestClassifier API (class_weight and probability prediction): https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html .

[5] XGBoost v3.4.1 parameter source: https://github.com/dmlc/xgboost/blob/v3.4.1/doc/parameter.rst ; Python API: https://xgboost.readthedocs.io/en/stable/python/python_api.html .

[6] XGBoost, Introduction to Boosted Trees: https://xgboost.readthedocs.io/en/stable/tutorials/model.html .

[7] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR 16, 321-357. https://doi.org/10.1613/jair.953 .

[8] Imbalanced-learn, Common pitfalls and recommended practices: https://imbalanced-learn.org/stable/common_pitfalls.html .

[9] Scania CV AB (2016). APS Failure and Operational Data for Scania Trucks, supplied data description and official train/test CSVs. Dataset record: https://archive.ics.uci.edu/dataset/421/aps+failure+at+scania+trucks .

Execution and format

Python 3.12.8; numpy 2.5.4, pandas 3.1.1, scikit-learn 1.9.1, xgboost 3.4.1, imbalanced-learn 0.14.2, matplotlib 3.11.2, joblib 1.6.0. CPU only; seed 552; estimator threads 4 and sequential folds. Saved full-data fits, probability outputs and fold search files supply every numeric model table. The isolated full run fitted 2 forests, 70 CV estimators, 2 selected final XGB models and 1 positive-alpha supplement using only raw CSVs and the inline notebook. The self-contained notebook has full-from-raw and fingerprint-validated quick modes. A clean-kernel quick run reproduced all five original final-model train/test/sensitivity predictions and recomputed metrics; saved CV summaries/selection were checked without refitting CV estimators. The earlier full snapshot executed 75 fits. OOB review subsequently identified invalid balanced OOB coverage, retained the old arithmetic, and fitted only a separate balanced_subsample forest. The revised notebook was executed in quick mode with independent original-model and OOB-audit fingerprints. Historical models, XGB/SMOTE and all 70 CV results remain unchanged.

The displayed code is real Python from submission_core.py; it includes data loading/imputation, all feature statistics, plots, common evaluation, both forests, fold-local SMOTE/Pipeline, alpha search/selection, final XGBoost fits and all computational exercises. RUN_TRAINING=False disables fitting only during PDF layout; the displayed fitting logic is exercised by full notebook execution. Optional code for environment discovery, caching, serialization, report layout and repeated validation remains in the project.

Homework6.pdf, the local August 24, 2026 syllabus and course introduction were checked. No explicit font-size or margin rule was found there. Body 10.4 pt, monospaced code 8.7 pt and 36-pt side margins are layout choices, not claimed course mandates. All content in the formal PDF counts toward its 20-page limit. Separately attached notebook/source pages are not added, and the notice does not require printing the entire project. Necessary implementation and results remain here; supporting code stays in the notebook/project.

Edition cross-check: Q2/Q3 use second-edition 6.6.3/6.6.5; Q4/Q5 and Q6 5.4.2/8.4.4/9.7.2 agree across checked editions. Q6 6.8.4 explicitly uses the first-edition ridge exercise, equivalent to second-edition 6.6.4. Full source-page evidence is retained. The remaining instructor interpretation issue concerns node-level logistic model trees versus standard XGBoost, accurately distinguished on page 7.